The JavaScript controls the canvas rendering, collision detection for mouse clicks, state management for active/potted balls, and dynamic UI updates via modal overlays.

Key Variables & Initial Setup

- **DOM References:** `canvas`, `ctx`, `output`, `modal`, `modalTitle`, and `historyList` capture existing HTML elements to manipulate them dynamically.
- `balls` **Array:** Holds all active ball objects currently rendered on the table.
- `potHistory` **Array:** Serves as a persistent log storing data about potted balls in order (`order`, `ballNumber`, `type`, `pocket`).

Ball Position Calculation (The Triangle Rack)

```
for (let col = 0; col < 5; col++) {
    for (let row = 0; row <= col; row++) {
        const x = startX + col * (BALL_RADIUS * 1.732);
        const y = startY + (row - col / 2) * (BALL_RADIUS * 2);
        // ...
    }
}
```

- **Trigonometric Offset (`1.732`):** $\sqrt{3} \approx 1.732$. This comes from equilateral triangle geometry, packing the columns tightly without overlapping.
- **Vertical Centering (`row - col / 2`):** Offsets each column vertically based on how many balls are in that column, centering the entire rack around `startY`.

Drawing the Canvas (`drawTable`)

The `drawTable()` function clears the screen and iterates over the remaining objects in `balls[]`:

- `ctx.save()` / `ctx.restore()`**:** Isolates styles (like shadow effects or clip masks) so they don't bleed into subsequent drawings.
- **Striped Ball Clipping:**

```
ctx.arc(ball.x, ball.y, BALL_RADIUS, 0, Math.PI * 2);
ctx.clip();
ctx.fillRect(ball.x - BALL_RADIUS, ball.y - BALL_RADIUS/2, BALL_RADIUS * 2, BALL_RADIUS);
```

- `ctx.clip()` turns the ball circle into a mask. Drawing a simple rectangle inside this mask automatically produces a clean curved stripe.

- **Text Rendering:** Writes the ball's number cleanly in the middle using `ctx.textAlign = "center"` and `ctx.textBaseline = "middle"`.
-

Click Detection & Coordinate Scaling

```
const rect = canvas.getBoundingClientRect();
const scaleX = canvas.width / rect.width;
const scaleY = canvas.height / rect.height;

const mouseX = (event.clientX - rect.left) * scaleX;
const mouseY = (event.clientY - rect.top) * scaleY;
```

- **Why Scaling Matters:** Since the CSS resizes the canvas to fit responsive screens, raw pixel values from `event.clientX` won't match internal canvas internal dimensions (`800x400`). `scaleX` and `scaleY` map the browser's click coordinates back to the internal canvas space.
-
- **Distance Formula (Hit Detection):**
-

- $$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$
- Calculated using `Math.sqrt((mouseX - ball.x)**2 + (mouseY - ball.y)**2)`. If this distance is less than or equal to `BALL_RADIUS`, the user clicked inside that ball.
-

State Updates & UI Flow

1. **Opening Modal:** When a ball is clicked, `selectedBall` stores its reference, and `modal.style.display = 'flex'` reveals the pocket overlay.
2.
3. **Selecting a Pocket (`selectPocket`):**
4.
    - Creates a new event record and pushes it to `potHistory`.
    -
    - **Filters out the ball:**
    -

```
balls = balls.filter(ball => ball.number !== selectedBall.number);
```

    - This immutably removes the potted ball from `balls[]`.
    -
    - Calls `drawTable()` to clear and re-render only the remaining balls.
    -
5. **Updating History Log (`updateHistoryUI`):** Clears `#historyList` and creates new `<li>` elements dynamically based on the updated `potHistory` array.